

**Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Monterrey**

Escuela de Ingeniería y Ciencias

Ingeniería en Ciencias de Datos y Matemáticas

Uso de álgebras modernas para seguridad y criptografía, MA2006B.604

**Reporte Técnico – Versión Completa
Gestor de Identidades y Firma Digital Solidaria**

Equipo 2:

Ana Lidia Hernández Díaz	A00838643
Ana Paula García Valverde	A01174572
María Fernanda Montoya López	A01743214
Javier Rojas Orrante	A01352213
Xavier Lago Hicks	A01403177

Profesores:

Raúl Gómez
Anas Wajid
Alberto F. Martínez

Proyecto: Firmas Solidarias – Tecnología Criptográfica para los Derechos Humanos

Socio Formador: Casa Monarca

Monterrey, Nuevo León. 07 de junio de 2026

Índice

1. Introducción	1
1.1. Descripción de la Problemática	1
2. Marco de referencia	1
2.1. Fundamentos de Criptografía de Clave Pública	1
2.1.1. Algoritmo RSA	1
2.1.2. Pilares de la Seguridad	2
2.2. Herramientas y Bibliotecas de Implementación	2
2.2.1. FastAPI como orquestador del Backend	2
2.2.2. JWT (JSON Web Tokens) como guardián de Sesiones	2
2.2.3. itsdangerous como el guardián de Sesiones	3
2.2.4. Encriptación	3
2.3. Referentes Operativos y Soluciones Existentes	3
2.4. Marco Legal y Ético	3
2.4.1. Derechos ARCO	4
2.4.2. Ley Federal de Protección de Datos Personales	4
2.4.3. Ética de la Ciberseguridad	4
3. Solución Propuesta y Arquitectura	4
3.1. Estructura de las Capas de Seguridad	4
3.1.1. Capa 1: Autenticación Bifurcada y Puerta de Entrada	5
3.1.2. Capa 2: Control de Acceso Basado en Roles (RBAC)	5
3.1.3. Capa 3: Validación Estructural de Datos y Contratos de Entrada	7
3.1.4. Capa 4: Persistencia y Bóveda Criptográfica (Datos en Reposo)	7
3.2. Flujo de una Petición a Través de las Capas (Caso de Éxito vs Error)	7
3.3. Justificación	8
3.3.1. Viabilidad de Infraestructura: Despliegue en Alojamiento Compartido (HostGator) vs. Nube Compleja (AWS)	8
3.3.2. Robustecimiento del Esquema de Autenticación mediante Bifurcación de Acceso (RBAC)	8
3.3.3. Adopción de Estándares Algorítmicos Criptográficos e Industriales	9
3.3.4. Filosofía del Ciclo de Vida: El Principio de Entrega Única (Single Delivery)	9
3.4. Implementación Técnica y Estructura del Software	10
3.4.1. BackEnd	10
3.4.2. FrontEnd	11
3.5. Conexión de los Componentes	12
4. Resultados	12
4.1. Estructura de la solución	13
4.2. Resultados del BackEnd y API	13
4.3. Resultados del FrontEnd e Interfaz	13
4.4. Resultados de la Solución Completa	14
4.5. Validación mediante Casos de Prueba	14
5. Introducción a la Segunda Etapa	16
6. Módulo de Firma Digital de Documentos	16
6.1. Fundamentos Criptográficos	16
6.1.1. Elección de RSA-PSS sobre PKCS1 v1.5	16
6.1.2. Generación del Folio Único de Verificación	16
6.2. Proceso de Firma Individual	16

6.3. Proceso de Co-firma (Multi-firma)	17
6.3.1. Arquitectura del Flujo Multi-firma	17
6.3.2. Modos de Firma	17
6.3.3. Estados del Documento Multi-firma	18
6.3.4. Notificaciones de Co-firma	18
6.4. Firma Masiva en Lote (Batch Signing)	18
6.4.1. Implementación del Procesamiento Batch	18
6.4.2. Metadatos Comunes del Lote	18
7. Portal de Verificación Pública	19
7.1. Arquitectura de Verificación sin Sesión	19
7.2. Modalidades de Verificación	19
7.2.1. Verificación por Folio	19
7.2.2. Verificación por Archivo	19
7.3. Respuestas del Sistema de Verificación	19
8. Módulo de Gestión de Beneficiarios	20
8.1. Motivación y Contexto	20
8.2. Modelo de Datos Encriptado	20
8.3. Arquitectura de Cifrado con Consultas (Searchable Encryption)	20
8.3.1. Cifrado de Texto (Fernet AES-256)	20
8.3.2. Digestos HMAC-SHA256 para Índices	21
8.4. Control de Acceso por Rol en el Módulo de Beneficiarios	21
8.5. Ciclo de Vida de un Registro de Beneficiario	21
8.6. Exportación con Integridad Criptográfica	21
8.7. Datos de Demostración	22
9. Sistema de Notificaciones y Salud del Certificado	22
9.1. Motivación	22
9.2. Arquitectura del NotificationService	22
9.2.1. Verificación de Expiración de Certificados	22
9.2.2. Verificación de Expiración de Cuentas	22
9.2.3. Notificaciones de Bloqueo por Intentos Fallidos	22
9.2.4. Indicadores Visuales en la Interfaz ("Salud del Certificado")	23
9.3. Buzón de Notificaciones	23
10. Sistema de Recuperación de Emergencia — Administrador Espejo	23
10.1. Problema de la Llave Única	23
10.2. Arquitectura del Patrón Espejo	23
10.3. Implementación del AdminRecoveryService	24
10.4. Sincronización del Admin Espejo	24
10.5. Flujo de Recuperación de Emergencia	24
11. Gestión Expandida de Usuarios — Panel de Administración	24
11.1. Ciclo de Vida Completo del Usuario	24
11.2. Vista del Certificado X.509	25
11.3. Exportación del Directorio de Usuarios	25
11.4. Exportación de la Bitácora de Auditoría	25
12. Motor DatabaseCrypto — Análisis Técnico Profundo	26
12.1. Derivación de Claves de Cifrado y Lookup	26
12.2. Cifrado Fernet: Detalles de Implementación	26
12.3. Migración de Datos Legados	26
13. Interfaz de Usuario — Diseño para Contextos Humanitarios	27
13.1. Principios de Diseño	27

13.2. Estructura de Navegación del Portal	27
13.3. Flujo de Autenticación Unificado	27
14. Configuración, Despliegue y Entorno de Producción	28
14.1. Variables de Entorno Críticas	28
14.2. Guardián de Producción (Production Guard)	28
14.3. Soporte Multi-Base de Datos	28
14.4. Containerización con Docker	28
15. Resultados de la Segunda Etapa	29
15.1. Funcionalidades Implementadas y Verificadas	29
15.2. Casos de Prueba Adicionales de la Segunda Etapa	29
15.3. Métricas Técnicas del Sistema Completo	30
16. Conclusiones Finales	30
16.1. Trabajo Futuro	31
17. Bibliografía Completa	31
18. Anexos	33
18.1. Tabla de casos de prueba	33

Abstract

Este reporte documenta el diseño e implementación de la primera etapa del reto “Gestor de Identidades” desarrollado para la organización Casa Monarca. El objetivo central fue establecer un sistema de manejo centralizado y seguro para las identidades digitales de los colaboradores, asegurando la protección de datos sensibles de la población migrante atendida. La solución técnica emplea una arquitectura de Control de Acceso Basado en Roles (RBAC) construida con el lenguaje Python y el framework FastAPI, utilizando una base de datos MySQL para la persistencia. El sistema gestiona el ciclo de vida completo de las identidades, incluyendo los procesos de alta, activación, revocación y expiración automática; y garantiza el no repudio mediante la generación de bitácoras de auditoría inmutables. Finalmente, el desarrollo se alineó con el marco legal mexicano de los Derechos ARCO y la Ley de Firma Electrónica Avanzada, priorizando un equilibrio ético entre la robustez criptográfica y la usabilidad para el personal operativo de la organización.

1. Introducción

El presente reporte brinda una solución a la problemática de gestión de identidades, la cual se desarrolla para la organización socioformadora Casa Monarca. El objetivo principal es brindar protección de la información sensible de las personas migrantes que reciben apoyo de la organización, esto por medio de un manejo centralizado y seguro de las identidades de los colaboradores a través de certificados digitales. El diseño de la solución busca alinear con el marco legal mexicano, cumpliendo con los Derechos ARCO y la Ley Federal de Protección de Datos Personales en Posesión de Particulares. El enfoque técnico busca priorizar la usabilidad y la eficiencia operativa, esto con el objetivo de mejorar la productividad sin exigir conocimientos técnicos avanzados al personal de la organización. Por otro lado, se asegura la integridad, autenticidad y el no repudio en la gestión de documentos y accesos a través de una arquitectura de Control de Acceso Basado en Roles (RBAC).

1.1. Descripción de la Problemática

Dado a que Casa Monarca maneja información sensible de personas migrantes en situación vulnerable, las principales anomalías que el sistema busca impedir son la suplantación de identidad, la alteración de datos y la falta de trazabilidad en el acceso a la información. De acuerdo a protocolos de seguridad modernos, se requiere un ciclo de vida completo para los usuarios donde exista un alta, baja, revocación y rastreo de estos.

2. Marco de referencia

2.1. Fundamentos de Criptografía de Clave Pública

La base de la solución reside en la criptografía asimétrica, la cual utiliza pares de claves, siendo la pública y la privada, para garantizar la seguridad en entornos de red. Es importante mencionar el uso de algoritmos criptográficos como el RSA.

2.1.1. Algoritmo RSA

Es un algoritmo que permite el intercambio seguro de información y la generación de firmas digitales, por lo cual es lo que garantiza el no repudio y la autenticidad. Además, es considerado el estándar para asegurar que solo el poseedor de la clave privada pueda descifrar información o generar firmas

digitales auténticas. Por medio de la generación de llaves, donde se crean dos números primos grandes y mediante operaciones matemáticas, se derivan la llave pública (n, e) y la privada (n, d) . Para cifrarlo el emisor toma el mensaje (convertido a número) y lo eleva a la potencia e módulo n y para decifrarlo, el receptor toma el criptograma y lo eleva a la potencia d módulo n para recuperar el mensaje original.

2.1.2. Pilares de la Seguridad

La implementación busca cumplir con los tres pilares de la seguridad, la confidencialidad, integridad y disponibilidad. La confidencialidad responde a la problemática del acceso no autorizado, en este contexto, que la información solo esté disponible únicamente a colaboradores de Casa Monarca. Esto resulta relevante debido a la información sensible que se maneja dentro de la organización y si llegara a las manos equivocadas se vulneraría los derechos de las personas migrantes.

Por otro lado la integridad, definida como el "estado de lo que está completo o tiene todas sus partes", garantiza que la información sea correcta y no modificada, lo cual implica que la información esté tal cual como fue creada y no exista algún tipo de alteración por parte de terceros. Una cualidad muy importante para este pilar es la autenticación, la cual identifica a la persona que creó la información. Por último, la disponibilidad garantiza que se pueda acceder a la información cuando se requiera. En este sentido, se busca que exista un balance entre este pilar y la confidencialidad, ya que a menudo suelen conflictuarse debido a que un cifrado complejo puede dejar de ser accesible.

2.2. Herramientas y Bibliotecas de Implementación

Para la construcción del sistema en Python, se seleccionaron herramientas que permiten aplicar estos conceptos matemáticos de forma eficiente.

2.2.0.1. La biblioteca cryptography como el motor criptográfico

Esta librería es el estándar de la industria en Python utilizado para realizar operaciones criptográficas y de firma digital. En el proyecto, su uso se centraliza en dos frentes críticos.

- **Infraestructura de Clave Pública (PKI):** Se utiliza para modelar la Autoridad Certificadora (CA) interna, permitiendo generar pares de llaves asimétricas RSA de 2048 bits con un exponente público de 65537. Asimismo, gestiona la construcción, codificación y firma de certificados digitales estándar X.509 para administradores y coordinadores.
- **Protección de Material Criptográfico:** Emplea el algoritmo de empaquetado seguro PKCS8 con el método BestAvailableEncryption (el cual aplica AES-256 según el OpenSSL local) para cifrar la llave privada del usuario con su contraseña antes de estar disponible para su descarga.

El hashing y verificación de contraseñas no utiliza librerías externas de terceros, sino que se ejecuta a través del módulo nativo hashlib.pbkdf2_hmac de Python, configurado de manera robusta bajo PBKDF2-HMAC-SHA256 con 120,000 iteraciones y sales de 16 bytes generadas por secrets.token_bytes.

2.2.1. FastAPI como orquestador del Backend

FastAPI se encarga de unir la seguridad con la lógica de negocio definiendo las rutas para el ciclo de vida del usuario: Alta, Baja, Revocación y Rastreo, por medio de la gestión de endpoints. Además, se utilizarán inyección de dependencias para validar automáticamente el actor que realiza una acción antes de procesar cualquier lógica, asegurando que solo usuarios autenticados lleguen a la base de datos.

Validación de Datos: Gracias a su integración nativa, asegura que toda información que entra al sistema cumpla con el formato correcto antes de ser procesada por los servicios criptográficos.

2.2.2. JWT (JSON Web Tokens) como guardián de Sesiones

El manejo de identidades digitales requiere una forma segura de transmitir quién es quién sin exponer credenciales constantemente. Es por esto que tras una autenticación exitosa (MFA), el sistema genera

un JWT firmado, este token viaja en cada petición del usuario y contiene sus privilegios y rol. Con el middleware de Autorización, el sistema revisa el JWT en cada paso para determinar si el rol asociado tiene el permiso necesario (ej. users:revoke) para ejecutar la acción solicitada.

2.2.3. itsdangerous como el guardián de Sesiones

El manejo de identidades digitales en la capa web no utiliza JWT (JSON Web Tokens), sino cookies de sesión firmadas criptográficamente para evitar la exposición constante de credenciales. Tras una autenticación exitosa, el servidor utiliza la clase `URLSafeTimedSerializer` provista por la biblioteca `itsdangerous` para generar un token compacto firmado con la clave secreta global (`session_secret`) y el salt del sistema. Este token se inyecta en el navegador a través de la cookie segura `cm_session`. En cada petición HTTP entrante, el servidor intercepta la cookie de forma obligatoria, verifica la integridad de su firma HMAC en tiempo constante y valida que no haya superado su tiempo de expiración máximo configurado de 8 horas.

2.2.4. Encriptacion

Para proteger la información almacenada en la base de datos, el proyecto utiliza la librería `cryptography`, específicamente `Fernet`, como mecanismo principal de cifrado simétrico. Su función es convertir los datos sensibles en texto cifrado antes de guardarlos y descifrarlos automáticamente cuando el sistema necesita leerlos. Esta lógica se implementa mediante las clases `EncryptedText` y `EncryptedJSON`, que funcionan como tipos personalizados de `SQLAlchemy`. Su propósito es que campos como nombres, correos, certificados, llaves privadas, contraseñas hashadas, registros de auditoría y metadatos no queden almacenados en texto plano dentro de la base de datos.

Además, el sistema usa herramientas como `hashlib`, `hmac` y `base64` para reforzar la protección y compatibilidad de los datos. `hashlib` permite derivar material criptográfico a partir de una clave secreta del sistema, mientras que `hmac` genera identificadores de búsqueda seguros para poder consultar ciertos campos cifrados sin revelar su valor original. `base64` se usa para transformar datos binarios en texto compatible con almacenamiento en base de datos. En conjunto, estas herramientas permiten que la base de datos conserve funcionalidad operativa, como búsquedas e índices, pero reduciendo el riesgo de exposición si alguien accede directamente a las tablas.

2.3. Referentes Operativos y Soluciones Existentes

Tras una exhaustiva investigación acerca de los referentes operativos y las soluciones existentes, se analizaron modelos de gestión de identidades para adaptar sus mejores prácticas al contexto de la ONG. El principal referente es el sistema del SAT, del cual se tomó como referencia su modelo de administración de contribuyentes en México, el cual utiliza certificados digitales para vincular una identidad legal con un usuario digital, asegurando la trazabilidad de las acciones. Asimismo, se exploró el sistema de gestión de roles de la plataforma Canvas, la cual administra la vista del portal dependiendo del nivel del usuario.

Como soluciones existentes, se investigó la gestión del Ciclo de Vida, que de acuerdo con IBM se encarga de monitorear los datos a nivel de archivo, gestionando los archivos en función del tipo, el tamaño y la antigüedad. De este modo, siguiendo los estándares de la literatura, el sistema implementa las etapas de Alta, Baja, Revocación y Rastreo de identidades digitales.

2.4. Marco Legal y Ético

La solución técnica responde a un entorno normativo y humano riguroso, asegurando que la protección de datos sensibles se realice bajo estándares éticos y legales mexicanos.

2.4.1. Derechos ARCO

El diseño del sistema garantiza el cumplimiento de los derechos de Acceso, Rectificación, Cancelación y Oposición del usuario sobre su identidad digital.

- **Acceso:** Se facilita a través del Portal de Autoservicio, donde cada colaborador puede visualizar su información personal, el estado de su cuenta y la vigencia de sus certificados activos.
- **Rectificación:** El Administrador dispone de herramientas integradas en el Dashboard para corregir nombres, correos electrónicos o modificar roles según cambien las responsabilidades del personal.
- **Cancelación y Oposición:** Estos derechos se materializan mediante el sistema de revocación manual y la expiración automática de cuentas. Al marcar a un usuario como *revoked* o *expired*, el sistema bloquea de forma inmediata e irreversible el acceso, garantizando que los datos no sean tratados más allá del tiempo autorizado por el titular.

2.4.2. Ley Federal de Protección de Datos Personales

La arquitectura de seguridad por capas cumple con las medidas técnicas exigidas por la legislación mexicana para proteger la información de poblaciones vulnerables. El uso de hashing robusto mediante **PBKDF2-HMAC-SHA256** con 120,000 iteraciones asegura que las contraseñas nunca se almacenen en texto plano. Además, el material criptográfico de mayor sensibilidad, como las llaves privadas, se somete a un cifrado simétrico **AES-256** antes de su distribución. Para cumplir con el principio de responsabilidad y trazabilidad, el *AuditService* registra de manera inmutable cada interacción crítica, documentando el actor, la acción, la dirección IP y el resultado, permitiendo auditorías forenses.

2.4.3. Ética de la Ciberseguridad

Se prioriza un equilibrio entre la robustez técnica y la usabilidad para el personal humanitario de Casa Monarca, evitando barreras tecnológicas que entorpezcan su misión. Esto se logra mediante una autenticación bifurcada: mientras que los roles con mayor privilegio (*ADMIN* y *COORDINADOR*) utilizan firmas digitales RSA-PSS y certificados X.509, los voluntarios y operativos acceden con un método simplificado de correo y contraseña. El diseño visual basado en semáforos de estado y la automatización de procesos complejos, como la sincronización del Administrador Espejo y la expiración por *end_date*, permiten que la organización mantenga un entorno seguro de forma autónoma.

3. Solución Propuesta y Arquitectura

El sistema implementa una arquitectura de seguridad por capas basada en los principios de defensa en profundidad y mínimo privilegio. Bajo este enfoque, la seguridad del sistema sigue una serie de filtros consecutivos y complementarios estructurados tanto a nivel de red como criptográfico y de aplicación. En este sentido, si un atacante logra vulnerar o evadir una de las capas exteriores, los controles de las capas subsiguientes detienen la progresión de la amenaza, protegiendo el núcleo del sistema donde reside la base de datos con la información sensible.

3.1. Estructura de las Capas de Seguridad

A continuación se detalla la función matemática, algorítmica y lógica de cada una de las capas que componen la arquitectura del Gestor de Identidades, desde el punto de entrada del cliente hasta la persistencia de los datos.

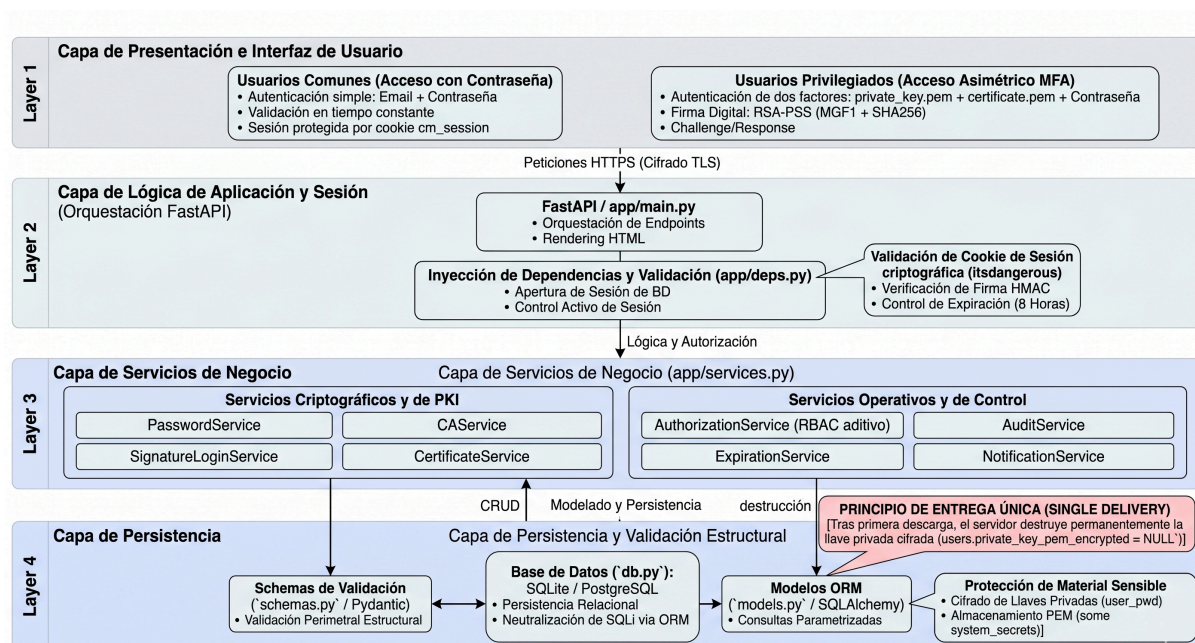


Figura 1: Arquitectura del sistema (elaboración propia)

3.1.1. Capa 1: Autenticación Bifurcada y Puerta de Entrada

Es el único punto de contacto con el exterior y actúa como el filtro perimetral de identidad aplicando un esquema estrictamente bifurcado según el nivel de riesgo del rol asignado.

Los usuarios comunes (OPERATIVO, VOLUNTARIO) se autentican mediante un esquema simple basado en algo que el usuario sabe, requiriendo únicamente su correo electrónico y su contraseña. El acceso es validado comparando el hash PBKDF2 de la contraseña ingresada contra el guardado en la base de datos utilizando comparaciones en tiempo constante (`hmac.compare_digest`). Por otro lado, para los usuarios privilegiados (ADMIN, COORDINADOR) se implementa un esquema obligatorio de dos factores basado en posesión criptográfica asimétrica. El usuario debe proporcionar su contraseña de descifrado y cargar dos archivos independientes en formato PEM: su archivo de llave privada (`private_key.pem`) y su certificado público (`certificate.pem`). El backend valida el número de serie, el correo incrustado en la extensión SAN (Subject Alternative Name), las ventanas de vigencia y la firma de la cadena de confianza. Finalmente, el servidor envía un desafío dinámico (challenge) compuesto por un nonce y una marca de tiempo, el cual debe ser firmado digitalmente mediante el algoritmo RSA-PSS (con esquema de máscara MGF1 y hashing SHA-256) utilizando la llave privada del usuario, validando matemáticamente la firma en el backend con la llave pública del certificado para autorizar la sesión.

3.1.2. Capa 2: Control de Acceso Basado en Roles (RBAC)

Una vez que el usuario demuestra su identidad de forma inequívoca en el perímetro del sistema (Capa 1), el servidor genera y gestiona una sesión web segura. La Capa 2 se encarga de limitar estrictamente qué acciones y recursos puede consumir ese usuario en función de sus competencias autorizadas, aplicando el principio de Mínimo Privilegio. Esta validación se divide en dos frentes complementarios: la autenticación de la sesión mediante cookies criptográficas y la autorización granular basada en roles (RBAC).

3.1.2.1. A. Mecanismo de Sesión Segura mediante `itsdangerous`

A diferencia de las arquitecturas que transmiten tokens expuestos como JWT independientes, el sistema delega el resguardo del estado del usuario en cookies de sesión firmadas criptográficamente en el servidor para mitigar riesgos de interceptación de credenciales. Tras un inicio de sesión exitoso, el

backend utiliza la clase `URLSafeTimedSerializer` provista por la biblioteca `itsdangerous` para instanciar un serializador acoplado a un secreto global de alta entropía (`session_secret`) y un salt específico denominado `"cm-session"`.

Este componente genera un token compacto e inmutable estructurado bajo el formato estándar de tres secciones de texto seguro:

$$\text{Token} = \text{payload_base64_urlsafe} \cdot \text{timestamp_base64} \cdot \text{firma_hmac_base64}$$

El cuerpo del mensaje (*payload*) transporta únicamente la información mínima de identificación del usuario mediante su identificador primario (`{"uid": user_id}`). La firma criptográfica se calcula utilizando un algoritmo HMAC-SHA256, lo que impide que un atacante externo altere los privilegios del token sin invalidar matemáticamente la firma completa.

Posteriormente, el servidor inyecta este elemento criptográfico en el navegador web del cliente bajo la cookie bautizada como `cm_session`, la cual cuenta con atributos restrictivos de seguridad activa:

- **`httponly=True`**: Restringe el acceso a la cookie exclusivamente al canal de transporte HTTP, impidiendo de forma absoluta que scripts maliciosos en el navegador (vectores de ataque XSS) puedan leer o exfiltrar el identificador de sesión.
- **`secure=True`**: Obliga al navegador a transmitir la cookie únicamente bajo conexiones cifradas mediante protocolos TLS/HTTPS en entornos productivos.
- **`samesite="lax"`**: Mitiga los vectores de ataque de Falsificación de Petición en Sitios Cruzados (CSRF), controlando que la cookie de sesión no sea adjuntada de manera automática en solicitudes originadas desde dominios externos o de terceros.
- **Expiración de la sesión (`_SESSION_MAX_AGE`)**: El ciclo de vida de la cookie está estrictamente acotado a un horizonte temporal de 8 horas (28,800 segundos), reduciendo drásticamente la ventana de oportunidad para el secuestro de sesiones en terminales físicos compartidos.

3.1.2.2. B. Intercepción de Peticiones y Control de Acceso Basado en Roles (RBAC)

Para asegurar que las políticas de acceso se ejecuten sin excepciones, el componente de inyección de dependencias nativo de FastAPI (`app/deps.py`) intercepta obligatoriamente la cookie de sesión en cada petición HTTP entrante. El backend lee el valor de `cm_session`, verifica la firma HMAC en tiempo constante y corrobora las marcas de tiempo frente al límite de expiración configurado. Simultáneamente, el interceptor realiza una validación lógica cruzada en la base de datos para constatar que la cuenta del actor se encuentre en un estado válido, denegando el flujo de manera inmediata si el estatus del usuario ha sido cambiado a `revoked` o `expired`.

Si la sesión es íntegra, toma el control el módulo `AuthorizationService`, el cual evalúa los permisos granulares (*scopes*) mediante un esquema RBAC de carácter estrictamente aditivo. La arquitectura del software rechaza el modelo de herencia de roles; en su lugar, la función `AuthorizationService.authorize(db, user, resource, action)` realiza un JOIN explícito en la base de datos relacional entre las entidades de usuarios, roles y la tabla asociativa `role_permissions`. De este modo, un usuario solo está facultado para ejecutar operaciones que se encuentren explícitamente mapeadas a su rol.

La matriz jerárquica de permisos granulares está limitada de la siguiente manera:

- **ADMIN**: Perfil de máxima confianza. Faculta el control absoluto sobre el catálogo de usuarios (`users: create, view, activate, revoke, change_role, change_expiration`), la consulta de la bitácora forense de auditoría (`audit: view`) y la activación de los protocolos de recuperación criptográfica de emergencia.
- **COORDINADOR**: Rol operativo privilegiado. Permite la visualización y edición avanzada de documentación interna (`documents: view, edit`), el monitoreo de operaciones de campo (`operations: view`) y la consulta de su material criptográfico personal (`certificates: view`).

- **OPERATIVO:** Perfil operativo básico. Restringido a la visualización pasiva de documentos (`documents: view`) y a la inserción o edición de registros de campo en operaciones generales (`operations: view, edit`).
- **VOLUNTARIO:** Rol de mínimo impacto. Limitado exclusivamente a la consulta y lectura de documentos autorizados para tareas humanitarias (`documents: view`).

Si un colaborador intenta consumir un servicio para el cual su rol no cuenta con un permiso explícito (por ejemplo, si un usuario con perfil **OPERATIVO** dispara una solicitud HTTPS hacia el endpoint de revocación `/users/revoke`), la inyección de dependencias detiene el procesamiento en el perímetro del backend. El sistema frena la petición de forma atómica, devuelve un código de estado HTTP 403 Forbidden y genera un registro síncrono inmutable a través del `AuditService` detallando la identidad del actor, la dirección IP de origen, el agente de usuario y el recurso comprometido, garantizando así la trazabilidad forense y el principio de no repudio.

3.1.3. Capa 3: Validación Estructural de Datos y Contratos de Entrada

Esta capa impide el procesamiento de datos maliciosos o mal formateados que busquen corromper la memoria del servidor o inyectar código en la persistencia. Antes de tocar las funciones de negocio o criptografía, los parámetros de entrada se someten a los modelos de validación de `schemas.py`. Aquí se inspecciona rigurosamente la entropía de las contraseñas, formatos estricto de correos electrónicos y tipos de datos. Si la entrada no cumple el contrato, el framework la rechaza en el perímetro. La interacción con la base de datos se realiza a través de objetos de Python en `models.py`. Al no utilizar concatenación de cadenas de texto plano para construir consultas SQL, el uso del ORM parametriza automáticamente todas las variables, neutralizando por completo el vector de ataque de Inyección SQL (SQLi).

3.1.4. Capa 4: Persistencia y Bóveda Criptográfica (Datos en Reposo)

Es la capa más profunda de la arquitectura, encargada de la persistencia de datos mediante el ORM `SQLAlchemy` mapeado hacia bases de datos relacionales `SQLite` (generando el archivo `identity_demo.db` en entornos locales) o `PostgreSQL` para producción. Para el alcance del entorno de demostración, los secretos globales de la organización (como la llave privada de la CA raíz y el certificado raíz) se almacenan como texto plano en formato PEM dentro de la tabla dedicada `system_secrets`. Sin embargo, el sistema protege activamente las llaves privadas de los usuarios privilegiados (**ADMIN** y **COORDINADOR**) guardándolas en un estado cifrado con la contraseña del propio usuario. Una vez que el usuario realiza la descarga de su credencial, el servidor ejecuta una rutina atómica que destruye el registro binario del servidor asignando el campo `private_key_pem_encrypted` a nulo, aplicando la filosofía de entrega única. Adicionalmente, el entorno se configura con la clase `Settings` de `Pydantic`, la cual actúa como salvaguarda activa impidiendo el arranque del servidor en entornos de producción si se detecta que las claves secretas conservan los valores por defecto.

3.2. Flujo de una Petición a Través de las Capas (Caso de Éxito vs Error)

Para visualizar la efectividad de la arquitectura, se puede analizar matemáticamente el recorrido de una solicitud HTTPS para la acción crítica de Revocación de un Usuario:

- 1) Recepción (Filtro Perimetral): El API Gateway en `app/main.py` recibe la petición HTTPS con el identificador del objetivo.
- 2) Evaluación de Capa 1 y 2 (Seguridad de Sesión): `app/deps.py` extrae la cookie de sesión. Descifra y verifica la firma HMAC del token; si el token fue manipulado o ya expiró por tiempo, la petición muere aquí devolviendo un error 401 Unauthorized. Si es correcto, extrae el perfil del actor.
- 3) Evaluación de Capa 2 (Autorización): El `AuthorizationService` verifica en la base de datos si el rol del actor cuenta con el permiso explícito para revocar. Si un rol no autorizado (como un voluntario) ejecuta

la petición, el sistema bloquea el flujo y responde 403 Forbidden. Si el actor es un “Administrador”, la petición avanza.

4) Evaluación de Capa 3 (Validación de Datos): Pydantic valida que el ID del usuario a revocar cumpla con el tipo de dato esperado.

5) Ejecución en Capa 4 (Persistencia): El servicio crítico actualiza el campo del usuario a revoked y confirma el cambio mediante un commit en SQLAlchemy, asegurando la integridad referencial de los datos.

6) Cierre de Auditoría: Una vez completada la transacción, el AuditService escribe un registro inmutable en la bitácora (audit_logs) documentando la IP de origen, el timestamp, el actor de la acción y el resultado exitoso, garantizando el principio de no repudio.

3.3. Justificación

El sistema fue concebido bajo la premisa de que una organización de derechos humanos como Casa Monarca, al manejar información altamente sensible de poblaciones vulnerables, requiere un nivel de protección equivalente al del sector bancario o gubernamental como el SAT, pero adaptado a una realidad operativa de bajo costo y alta usabilidad. Se detallan las decisiones técnicas que se realizaron, las cuales se justifican bajo los supuestos propuestos a continuación.

3.3.1. Viabilidad de Infraestructura: Despliegue en Alojamiento Compartido (HostGator) vs. Nube Compleja (AWS)

Una de las decisiones más críticas en el diseño de software es el entorno de despliegue. Aunque la tendencia industrial actual favorece el uso de proveedores de nube hiperescalables como AWS, Azure o GCP, para el contexto de Casa Monarca se optó por un despliegue en un entorno de hosting compartido tradicional (HostGator) utilizando scripts de automatización vía SSH.

Esto se debe principalmente para reducir costos operativos, ya que las organizaciones no gubernamentales (ONG) operan bajo presupuestos estrictos y donaciones variables. Los esquemas de precios de AWS se basan en el consumo como ancho de banda, lecturas/escrituras de base de datos, tiempo de cómputo de instancias EC2 y almacenamiento, lo que introduce una alta volatilidad financiera y el riesgo de cobros imprevistos debido a picos de tráfico o ataques de denegación de servicio (DoS). HostGator, en contraste, ofrece una estructura de costos fijos mensuales o anuales sumamente bajos, permitiendo a la organización planificar financieramente a largo plazo sin desviar fondos de su misión humanitaria. Otra consideración es la simplicidad en el mantenimiento, debido a que el uso de infraestructuras en la nube complejas exige la adopción de herramientas de orquestación, contenedores y automatización pesada como Docker, Kubernetes, Ansible o Terraform. Mantener esta cadena de herramientas (toolchain) requiere personal de ingeniería altamente especializado en DevOps de forma permanente. Al empaquetar el sistema como una aplicación Python/FastAPI nativa capaz de ejecutarse sobre un entorno básico con scripts SSH automatizados, se elimina la dependencia de un departamento de TI avanzado. La organización puede mantener el sistema bajo un modelo de administración simplificado, garantizando la continuidad operativa a lo largo de los años sin caer en la obsolescencia por falta de personal técnico.

3.3.2. Robustecimiento del Esquema de Autenticación mediante Bifurcación de Acceso (RBAC)

La arquitectura no aplica las mismas medidas de seguridad a todos los usuarios, sino que implementa una separación estricta de mecanismos de autenticación según el nivel de privilegio y la superficie de riesgo del rol que se asignó. De esta manera, se mitiga el phishing y se evitan que las credenciales tengan algún tipo de vulnerabilidad en roles críticos.

Los ataques de ingeniería social y phishing son responsables de la gran mayoría de las brechas de seguridad a nivel mundial. Si un atacante logra robar la contraseña de un usuario común (como un voluntario), el impacto se autolimita gracias al Control de Acceso Basado en Roles (RBAC), el cual restringe sus permisos exclusivamente a visualizaciones de bajo impacto. Por esta razón, para los roles OPERATIVO y VOLUNTARIO, un esquema de correo y contraseña robustecida con PBKDF2-HMAC-SHA256 e inyección de cookies seguras (HttpOnly, SameSite=Lax) es técnicamente proporcional y suficiente.

Sin embargo, para los roles con capacidad de alterar la configuración del sistema o exfiltrar bases de datos completas (ADMIN y COORDINADOR), las contraseñas tradicionales son un mecanismo de defensa inaceptable. Al obligar a estos roles a autenticarse mediante un protocolo de desafío-respuesta con llaves asimétricas RSA de 2048 bits y certificados X.509, la seguridad pasa a depender estrictamente de algo que el usuario posee. En este sentido, un atacante externo, incluso si intercepta la contraseña de descifrado local del coordinador, no podrá iniciar sesión si no posee el archivo digital .pem, reduciendo a cero el riesgo de intrusiones por ataques de diccionario o phishing remoto.

3.3.3. Adopción de Estándares Algorítmicos Criptográficos e Industriales

La selección del ecosistema tecnológico y criptográfico responde a estándares internacionales avalados por agencias de ciberseguridad como el NIST y el cumplimiento normativo del marco legal mexicano. Por ejemplo, la elección de FastAPI como framework principal responde tanto a su alto rendimiento en milisegundos como a su integración nativa con la validación de datos a través de Pydantic. Esto garantiza que cualquier intento de inyección de desbordamiento de búfer o envío de parámetros corruptos sea filtrado en el perímetro de la API.

Por su parte, SQLAlchemy funciona como una capa de abstracción sobre MySQL. Al mapear las entidades de seguridad como objetos de Python y parametrizar de forma obligatoria todas las consultas a las tablas (users, audit_logs, etc.), se elimina por completo la posibilidad de ataques de Inyección SQL (SQLi), una de las técnicas más comunes utilizadas por atacantes para secuestrar bases de datos.

Se prefirió el uso de RSA de 2048 bits por su altísima compatibilidad con almacenes de certificados estándar. Para el inicio de sesión se implementó obligatoriamente el acolchado PSS (Probabilistic Signature Scheme) en lugar del clásico PKCS1 v1.5. PSS introduce una sal aleatoria en cada operación de firma, lo que significa que el mismo desafío matemático firmado dos veces generará firmas completamente distintas, protegiendo al sistema contra ataques de criptoanálisis por texto elegido y ataques de repetición (replay attacks). Por otro lado, el uso de AES-256 (Advanced Encryption Standard con llaves de 256 bits) provisto por Pycryptodome / cryptography asegura que los datos en reposo dentro de MySQL cuenten con cifrado de grado militar. Si un tercero sustrae físicamente los discos duros del servidor o exfiltra los archivos de respaldo del hosting, se topará con bloques de datos indistinguibles del ruido aleatorio, blindando la confidencialidad de la población migrante atendida.

3.3.4. Filosofía del Ciclo de Vida: El Principio de Entrega Única (Single Delivery)

El almacenamiento permanente de llaves privadas en un servidor centralizado es propenso a amenazas, ya que si el servidor sufre una intrusión y el atacante obtiene privilegios de superusuario (root), podría descifrar las llaves de todos los coordinadores de la organización. Para neutralizar este riesgo estructural, el sistema implementa la política de Entrega Única. El servidor funciona únicamente como una fábrica temporal de material criptográfico. En el instante en que el administrador genera un usuario y descarga su llave privada empaquetada, el backend ejecuta una rutina que destruye el registro binario en la base de datos, dejando el campo private_key_pem_encrypted estrictamente en nulo.

A partir de ese segundo, la llave privada solo existe en el dispositivo físico del coordinador. Esta decisión de diseño traslada la responsabilidad de la custodia al eslabón final y garantiza de forma matemática el principio de No Repudio: si se registra una acción firmada digitalmente en la bitácora de auditoría, la organización tiene la certeza absoluta de que solo pudo haber sido ejecutada por el poseedor de

esa llave física, imposibilitando que incluso los propios administradores del sistema puedan falsificar la firma de un coordinador.

3.4. Implementación Técnica y Estructura del Software

3.4.1. BackEnd

El backend, orquestado mediante FastAPI, actúa como un API Gateway interno que intercepta todas las peticiones HTTPS, gestionando la serialización de datos y evitando que errores técnicos filtren información sensible. Los componentes y servicios críticos (app/services.py) se dividen en las siguientes responsabilidades:

Servicio de Inicio de Sesión Criptográfico (SignatureLoginService)

Es el componente backend con mayor rigurosidad criptográfica. Es el encargado de procesar la autenticación de dos factores para los roles privilegiados (ADMIN y COORDINADOR).

- Validación ASN.1 del Certificado: Parsea en memoria el archivo certificate.pem cargado por el usuario, comprobando el número de serie único almacenado en la base de datos y corroborando que el tiempo del sistema esté dentro de las ventanas not_valid_before_utc y not_valid_after_utc.
- Verificación de Identidad Cruzada: Extrae el campo Subject Alternative Name (SAN) con formato de correo electrónico (RFC822Name) y valida que coincida exactamente con la identidad del usuario que solicita el acceso.
- Procesamiento del Desafío (Challenge-Response): Genera un nonce dinámico (un bloque de datos único que incluye el ID del usuario y un timestamp). El servicio utiliza la llave pública extraída del certificado para verificar la firma digital enviada por el cliente (calculada con el algoritmo asimétrico RSA-PSS con máscara MGF1 y SHA-256). Si la verificación matemática pasa, se demuestra la posesión física de la llave privada y se autoriza la sesión.

Servicio de Autoridad Certificadora Interna (CertificateAuthorityService)

Es el núcleo de confianza (Root of Trust) de la organización.

- Gestión de la CA Raíz: Se encarga de inicializar de manera persistente en el almacén de secretos del sistema (system_secrets) el par de llaves asimétricas y el certificado X.509 autofirmado de la CA con una vigencia extendida de 10 años.
- Emisión de Credenciales de Coordinadores: Utiliza la llave privada de la CA para firmar digitalmente los certificados emitidos a los coordinadores, estableciendo la propiedad issuer_name con los datos de la CA del sistema, inyectando las extensiones obligatorias de restricción BasicConstraints(ca=False) y los usos permitidos (KeyUsage.digital_signature y KeyUsage.key_encipherment).

Servicio de Auditoría e Inmutabilidad (AuditService)

Este es el encargado del cumplimiento del principio de No Repudio dentro del sistema.

-Registro de Eventos Críticos: Captura de forma síncrona metadatos detallados de cada petición exitosa o fallida (como intentos de login por contraseña, logins criptográficos, revocaciones de usuarios, accesos denegados o expiraciones automáticas).

-Persistencia Segura: Escribe los eventos en la tabla relacional audit_logs, registrando de manera estructurada campos como: actor_user_id, target_user_id, action, resource, result, ip_address y user_agent. Este flujo cierra el ciclo de seguridad, permitiendo una trazabilidad forense completa si se detectan anomalías.

Servicio de Expiración Automática (ExpirationService)

Es el proceso asíncrono que actúa de manera proactiva para reducir la carga administrativa y mejorar la seguridad de la plataforma.

- **Evaluación de Vigencia Lógica:** Examina periódicamente el campo `end_date` de los usuarios cuyo estatus actual sea `active`.
- **Inactivación Atómica:** Si la marca de tiempo del servidor supera la fecha de finalización configurada, el servicio cambia automáticamente el estatus del usuario a `expired`, bloqueando inmediatamente cualquier intento de uso de sus tokens o certificados activos.

3.4.2. FrontEnd

La interfaz web está diseñada para permitir al personal humanitario de Casa Monarca interactuar ágilmente con los servicios criptográficos subyacentes sin enfrentarse a barreras o complejidades tecnológicas complejas. Los módulos más relevantes en la presentación son:

Panel de Control de Identidades (Dashboard de Administración)

Es la consola centralizada de supervisión reservada para el rol de ADMIN.

- **Control Visual de Estados:** Implementa códigos de color dinámicos basados en semáforos para los estados de los colaboradores (`active` → verde, `pending` → amarillo, `revoked` → rojo, `expired` → gris), facilitando auditorías visuales e inspecciones rápidas del estatus del personal por parte del administrador.
- **Ocultamiento de Acciones Críticas por RBAC:** La interfaz web renderiza los componentes interactivos de manera condicional. Los botones para funciones altamente sensibles, tales como “Revocar Usuario”, “Cambiar Rol” o “Crear Usuario”, solo se inyectan en el DOM si el token de sesión descifrado confirma el rol jerárquico de Administrador. Esto limita la superficie de ataque y previene el error humano por parte de operadores con menores privilegios.

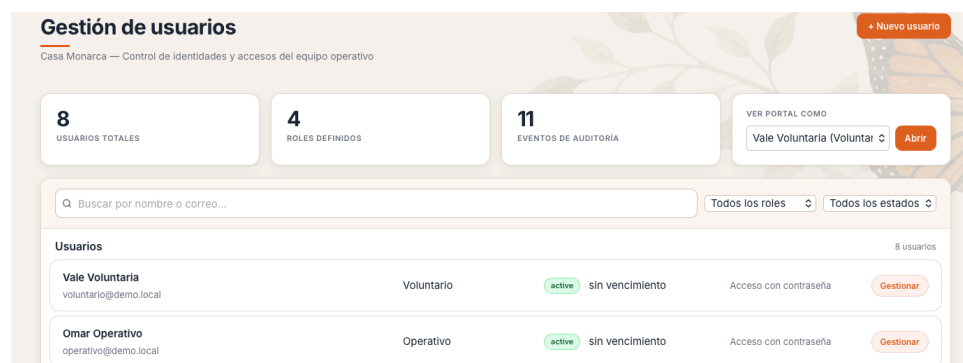


Figura 2: Panel de Control del administrador (elaboración propia)

Módulo de Autenticación Bifurcado (Formularios de Inicio de Sesión)

Presenta una interfaz de acceso adaptada al nivel de riesgo del usuario.

- **Formulario Estándar:** Para voluntarios y operativos, ofrece campos limpios de entrada de texto para correo electrónico y contraseña.
- **Formulario de Carga Criptográfica:** Para roles privilegiados, se transforma en una interfaz de carga de archivos de arrastrar y soltar (drag-and-drop). Solicita explícitamente tres componentes esenciales: el archivo de certificado (`certificate.pem`), el archivo de llave privada (`private_key.pem`) y la contraseña de paso utilizada para descifrar localmente la llave empaquetada.

Archivo de acceso

Solo para Administrador y Coordinador

**Arrastra tu archivo aquí**
o selecciona un archivo**Certificado de identidad**

Solo para Administrador y Coordinador

**Arrastra tu certificado aquí**
o selecciona un archivo

Figura 3: Solicitud de credenciales (elaboración propia)

Portal de Autoservicio y Descarga de Certificados (User Portal)

Módulo seguro al que tienen acceso los coordinadores validados.

- **Descarga Única de Llave Privada:** Aloja el componente interactivo que permite al usuario descargar por única vez la llave privada cifrada generada por el servidor. La interfaz despliega advertencias críticas informando que el archivo se borrará permanentemente de los servidores tras confirmar la descarga, reforzando la cultura de custodia digital.
- **Consulta de Certificado Propio:** Permite al colaborador visualizar las extensiones de su certificado X.509 activo, verificar su fecha límite de vigencia y descargar el certificado público tantas veces como sea necesario para la firma de documentos institucionales.



Figura 4: Portal del Usuario (elaboración propia)

3.5. Conexión de los Componentes

La arquitectura sigue un flujo de comunicación directo y seguro empezando por la orquestación donde FastAPI recibe las peticiones desde el FrontEnd mediante formularios HTML y parámetros de consulta. Posteriormente se requerirá la validación de identidad, donde antes de procesar cualquier lógica, el sistema utiliza dependencias (deps.py) para inyectar la sesión de la base de datos y validar quien realiza la acción. Los datos se almacenan en un esquema relacional definido en SQL que vincula usuarios con roles y permisos a través de llaves foráneas, garantizando la integridad referencial de toda la estructura de identidades. Cada interacción exitosa o denegada en el FrontEnd dispara una llamada al AuditService en el BackEnd, cerrando el ciclo de seguridad y control.

4. Resultados

La implementación del Gestor de Identidades dio como resultado una plataforma funcional que integra seguridad criptográfica con una gestión administrativa ágil. Los resultados se dividen en los logros alcanzados en el servidor, la interfaz y la validación operativa.

4.1. Estructura de la solución

```
"RetoCriptografia-Equipo2/
├── app/
│   ├── __init__.py
│   ├── config.py
│   ├── db.py
│   ├── deps.py
│   ├── main.py
│   ├── main2.py
│   ├── models.py
│   ├── schemas.py
│   └── services.py
├── docs/
│   └── USO_Y_FUNCIONAMIENTO.md
├── sql/
│   └── schema.sql
├── .env.example
├── .gitignore
├── pyproject.toml
└── README.md", block: true
```

4.2. Resultados del BackEnd y API

Se logró consolidar una infraestructura robusta basada en FastAPI, capaz de procesar peticiones de identidad en milisegundos. Se consolidó el servicio SignatureLoginService para usuarios privilegiados (ADMIN y COORDINADOR). Este procesa y valida de forma exitosa firmas digitales generadas con el esquema probabilístico avanzado RSA-PSS (utilizando máscara MGF1 y SHA-256) sobre un challenge dinámico generado por el servidor. La verificación de los hashes de contraseñas (derivadas mediante PBKDF2-HMAC-SHA256 con 120,000 iteraciones) se ejecuta mediante hmac.compare_digest. Esto garantiza comparaciones en tiempo constante, anulando la posibilidad de que un atacante deduzca credenciales analizando microsegundos de respuesta.

El AuditService registra de manera síncrona metadatos estructurados (IP, timestamp, actor, acción y recurso) en la tabla audit_logs ante cualquier interacción crítica, blindando la trazabilidad del sistema, lo cual cumple con el principio de no repudio. Además, el backend se integró exitosamente con MySQL/SQLite donde la información sensible y el material PKI temporal se almacenan bajo el estándar de cifrado simétrico AES-256-CBC, cumpliendo con el marco legal mexicano de protección de datos.

4.3. Resultados del FrontEnd e Interfaz

La interfaz web construida permite al personal operativo y humanitario de Casa Monarca interactuar de forma intuitiva con el backend criptográfico sin requerir conocimientos técnicos avanzados. Se pudo establecer un control visual de los estados, donde se implementó un código de colores dinámico basado en semáforos para la supervisión rápida del estatus de los colaboradores (active: verde, pending: amarillo, revoked: rojo, expired: gris). Además, gracias a la inyección de dependencias y la lectura del token de sesión, los componentes interactivos críticos (botones como “Revocar”, “Cambiar Rol” o “Crear Usuario”) se ocultan del DOM si el usuario no posee el rol de Administrador, limitando drásticamente la superficie de error humano.

Por otro lado, se obtuvo una interfaz de inicio de sesión bifurcada que despliega una entrada simple de credenciales para usuarios comunes, y un módulo de carga de archivos de arrastrar y soltar (drag-and-drop) que solicita el certificado (.pem), la llave privada (.pem) y su clave de descifrado para accesos privilegiados. Por último, el portal de auto servicio de descaega única permitió un espacio seguro donde

los coordinadores pueden descargar su certificado público y realizar la descarga única de su llave privada empaquetada bajo el estándar PKCS8

4.4. Resultados de la Solución Completa

Al integrar de extremo a extremo el BackEnd criptográfico con la interfaz del FrontEnd, se garantiza, en primer lugar, el principio de entrega única, cuando el Administrador genera un usuario privilegiado y este descarga su llave privada por primera vez, el sistema ejecuta una rutina atómica que destruye el registro binario en el servidor, asignando el campo `private_key_pem_encrypted` a nulo. A partir de ese instante, la llave privada solo existe en el dispositivo físico del coordinador, blindando al sistema contra exfiltraciones masivas si la base de datos central es comprometida.

Por otro lado, el `ExpirationService` evalúa de forma asíncrona el campo `end_date` de los usuarios activos. Si la vigencia de la cuenta ha caducado, cambia su estado a `expired`. El backend rechaza automáticamente cualquier intento de autenticación si las vigencias del certificado X.509 (`not_after_utc`) y la fecha lógica de la base de datos no se encuentran perfectamente sincronizadas, resolviendo eficientemente la gestión de revocaciones. La clase `Settings` (basada en `Pydantic`) actúa como una salvaguarda activa al validar la configuración del sistema. Si el software se despliega con el entorno marcado como `production`, el validador estructural genera un fallo crítico e impide que el servidor backend arranque si la clave `SESSION_SECRET` conserva su valor por defecto de desarrollo, forzando la seguridad de la infraestructura desde el diseño.

4.5. Validación mediante Casos de Prueba

Para asegurar la fiabilidad de la solución, se sometió al sistema a una batería exhaustiva de pruebas funcionales y de seguridad. Se validaron procesos críticos como el Alta de usuario con datos válidos, la denegación de accesos por contraseñas incorrectas o por usuarios no activos (estados `pending` o `revoked`), y la correcta visualización de la bitácora de auditoría por parte del administrador. Estas pruebas confirmaron que el sistema rechaza correctamente operaciones duplicadas o incompletas y que responde con los códigos de error adecuados (401 y 403) ante intentos de acceso no autorizados, garantizando la integridad del flujo de identidades.

A continuación se presenta un resumen de los casos de prueba ejecutados exitosamente:

Caso de Uso	Descripción	Resultado Esperado	Estatus
Alta Válida	Creación de usuario con datos únicos.	Usuario creado y certificado emitido.	Exitoso
Login Incorrecto	Intento de acceso con credenciales erróneas.	Rechazo de login y registro en auditoría.	Exitoso
Revocación	Cambio de estado de un usuario activo a revocado.	Acceso denegado inmediatamente tras el cambio.	Exitoso
Expiración	Cambio automático al llegar a <code>end_date</code> .	Usuario marcado como <code>expired</code> sin intervención.	Exitoso
Auditoría	Consulta de bitácora como administrador.	Despliegue de eventos recientes de seguridad.	Exitoso

Tabla 1: Resumen de validaciones (elaboración propia)

Etapa 2: Versión Completa del Sistema

5. Introducción a la Segunda Etapa

La primera etapa estableció la infraestructura criptográfica de identidades: autenticación, certificados, RBAC y auditoría. La segunda etapa extiende ese cimiento en cinco dimensiones: el módulo de Firma Digital de Documentos, el Portal de Verificación Pública, el Módulo de Gestión de Beneficiarios, el Sistema de Notificaciones y Salud del Certificado y la Recuperación de Emergencia mediante Administrador Espejo.

Estas extensiones consolidan el sistema en una plataforma de derechos humanos digitalmente robusta.

6. Módulo de Firma Digital de Documentos

6.1. Fundamentos Criptográficos

El módulo de firma digital se construye sobre la infraestructura PKI establecida en la Etapa 1, aplicando el principio de cero almacenamiento: el documento original nunca reside en el servidor de Casa Monarca. Solo su huella criptográfica y la firma digital son persistidas.

El proceso matemático central es:

$$H = \text{SHA-256}(D)$$

$$\sigma = \text{RSASSA-PSS}_{\text{sign}(H, sk_U, \text{MGF1-SHA256}, \text{salt}_{\text{len}}=32)}$$

donde D es el contenido binario del documento, H es el hash de 256 bits (32 bytes), sk_U es la llave privada RSA-2048 del usuario firmante y σ es la firma digital de 256 bytes que se almacena junto al folio de verificación.

6.1.1. Elección de RSA-PSS sobre PKCS1 v1.5

El sistema emplea RSA-PSS (Probabilistic Signature Scheme) en lugar del clásico PKCS1 v1.5 debido a que PSS introduce una sal de 32 bytes en cada operación de firma. El mismo documento firmado dos veces por el mismo usuario genera firmas completamente distintas, protegiendo contra análisis de criptoanálisis por texto elegido. Además, la combinación (documento, sal, timestamp) hace inviables los *replay attacks* y el PSS tiene reducción de seguridad formal al problema RSA, a diferencia de PKCS1 v1.5 cuya seguridad no está formalmente demostrada.

6.1.2. Generación del Folio Único de Verificación

Tras la firma, el sistema genera un Folio Único de Verificación con la estructura:

CM- {YYYY} -DOC- {XXXX}

donde {YYYY} es el año de emisión y {XXXX} es un identificador incremental con relleno de ceros. El folio actúa como referencia pública que permite la verificación posterior sin exponer ningún dato sensible del firmante ni del contenido.

6.2. Proceso de Firma Individual

El flujo completo para la firma de un documento estándar involucra las cuatro capas de seguridad del sistema:

1. *Carga del archivo*: el usuario selecciona o arrastra el archivo en la interfaz (formatos: PDF, DOC, DOCX, TXT, ODT; tamaño máximo: 20 MB). El archivo se transmite al servidor en el cuerpo de la petición HTTP multipart/form-data.
2. *Extracción de la huella digital*: el servidor extrae únicamente el hash SHA-256 del contenido binario del archivo. El archivo original es descartado inmediatamente de la memoria del servidor sin persistirse en ningún almacén.
3. *Recuperación de la llave privada*: el servidor recupera la llave privada del firmante de la tabla `system_secrets` (para administradores que aún no han entregado su llave) o la sesión criptográfica verificada (para coordinadores cuya llave ya fue entregada y que la presentaron en la autenticación Challenge-Response de Capa 1).
4. *Operación de firma*: se ejecuta la firma RSA-PSS-SHA256 sobre el hash extraído, generando la firma σ de 256 bytes.
5. *Persistencia del artefacto*: se almacena únicamente el hash, la firma, el folio generado, los metadatos del documento (título, tipo, vigencia) y la referencia al firmante. El documento original nunca se guarda.
6. *Auditoría*: AuditService registra el evento `document_signed` con `actor_user_id`, folio, hash (primeros 16 caracteres) y timestamp.
7. *Respuesta al cliente*: el sistema entrega el folio único al firmante con instrucciones para compartirlo junto al archivo original cuando requiera verificación.

6.3. Proceso de Co-firma (Multi-firma)

Cuando un documento requiere la autorización de más de un coordinador o administrador para tener validez institucional, se activa el flujo de co-firma:

6.3.1. Arquitectura del Flujo Multi-firma

El sistema implementa una estructura de cadena de firmas donde cada firma criptográfica opera de forma independiente sobre el mismo hash del documento original:

$$\begin{aligned}
 H &= \text{SHA-256}(D) \\
 \sigma_1 &= \text{RSASSA-PSS}_{\text{sign}(H, \text{sk}_{\{U_1\}})} \\
 \sigma_2 &= \text{RSASSA-PSS}_{\text{sign}(H, \text{sk}_{\{U_2\}})} \\
 &\dots \\
 \sigma_n &= \text{RSASSA-PSS}_{\text{sign}(H, \text{sk}_{\{U_n\}})}
 \end{aligned}$$

Cada firma es independiente, verificable por separado, y el documento solo adquiere estado VÁLIDO cuando **todas** las firmas requeridas han sido recibidas y verificadas.

6.3.2. Modos de Firma

- *Modo paralelo*: todos los co-firmantes reciben notificación simultánea y pueden firmar en cualquier orden. El documento pasa a estado VÁLIDO cuando se completa el conjunto.
- *Modo secuencial*: el sistema obliga a que los usuarios firmen en un orden específico definido en el momento de creación del flujo. Un firmante no puede actuar hasta que el anterior haya completado su firma. Este modo garantiza jerarquías de aprobación.

6.3.3. Estados del Documento Multi-firma

Estado	Indicador	Significado
PENDIENTE	Amarillo	Al menos una firma requerida aún no ha sido recibida
PARCIAL	Naranja	Algunas firmas recibidas pero no todas las requeridas
VÁLIDO	Verde	Todas las firmas verificadas; documento con validez institucional
RECHAZADO	Rojo	Un firmante rechazó el documento; proceso cancelado
EXPIRADO	Gris	La vigencia del documento caducó antes de completarse

6.3.4. Notificaciones de Co-firma

El sistema NotificationService genera automáticamente alertas de co-firma pendiente en el buzón de notificaciones de cada co-firmante designado. En el modo secuencial, la notificación solo se envía al siguiente firmante en la cadena una vez que el firmante anterior ha completado su participación, respetando el orden de aprobación definido.

6.4. Firma Masiva en Lote (Batch Signing)

Para escenarios operativos donde al final del día se deben firmar múltiples constancias del mismo tipo (por ejemplo, constancias de asistencia a un mismo programa), el sistema provee el módulo de firma en lote:

6.4.1. Implementación del Procesamiento Batch

El módulo acepta hasta 20 archivos simultáneamente del mismo tipo de documento. El procesamiento es atómico desde la perspectiva del usuario pero secuencial internamente:

```

PARA cada archivo f_i en el lote:
    H_i = SHA-256(contenido(f_i))
    σ_i = RSASSA-PSS(H_i, sk_usuario)
    folio_i = generar_folio(año_actual, contador_siguiete)
    GUARDAR(H_i, σ_i, folio_i, metadata_comun)
    REGISTRAR_AUDITORIA(actor, "batch_signed", folio_i)
FIN PARA

```

La llave privada del firmante se descripta una única vez al inicio del proceso y se mantiene en memoria durante el procesamiento del lote completo, liberándose inmediatamente al finalizar. Esto minimiza el número de operaciones de descriptación y reduce la exposición del material criptográfico.

6.4.2. Metadatos Comunes del Lote

Para un lote de firma masiva, los siguientes metadatos se definen una vez y aplican a todos los documentos del conjunto:

- *Tipo de documento*: categoría institucional (Constancia, Carta, Oficio, Reporte)
- *Vigencia*: número de días de validez del documento desde la fecha de firma
- *Firmante*: identidad del coordinador o administrador que ejecuta el lote

7. Portal de Verificación Pública

7.1. Arquitectura de Verificación sin Sesión

El portal de verificación está diseñado como un servicio completamente público, accesible sin autenticación, que permite a cualquier entidad externa (autoridades, instituciones, otros albergues) comprobar la autenticidad de un documento emitido por Casa Monarca.

La verificación se basa en el principio de la criptografía asimétrica: cualquiera con acceso a la llave pública del firmante puede verificar una firma, pero solo el poseedor de la llave privada pudo haberla generado.

7.2. Modalidades de Verificación

7.2.1. Verificación por Folio

El verificador ingresa el Folio Único de Verificación (ej. CM-2026-D0C-0042) y el sistema:

1. Recupera de la base de datos el hash SHA-256 del documento original, la firma digital σ , el certificado del firmante y los metadatos (fecha, tipo, vigencia, estado).
2. Comprueba que el folio exista y que la vigencia del documento no haya expirado.
3. Presenta al verificador: quién firmó el documento, con qué rol, en qué fecha, y si sigue vigente o ha sido marcado como expirado.

El verificador por folio proporciona autenticidad administrativa (confirma que el documento fue registrado en el sistema de Casa Monarca) pero no puede verificar que el archivo físico recibido no fue alterado después de la firma.

7.2.2. Verificación por Archivo

Esta es la verificación criptográficamente completa. El verificador sube el archivo digital recibido y el sistema:

1. Calcula en tiempo real: $H' = \text{SHA-256}(D')$ donde D' es el archivo recibido.
2. Recupera de la base de datos el hash original H y la firma σ asociada al folio.
3. Verifica la firma: comprueba que $\text{RSASSA-PSS}_{\text{verify}(H, \sigma, \text{pk}_U)} = \text{VERDADERO}$.
4. Compara $H' = H$:
 - Si $H' = H$: el documento no fue alterado. La firma es válida.
 - Si $H' \neq H$: se emite la alerta "Documento alterado", indicando que el archivo fue modificado después de la firma, incluso si el cambio fue de un solo bit.

La sensibilidad del SHA-256 al efecto avalancha garantiza que cualquier modificación, por mínima que sea, produce un hash completamente diferente, haciendo la detección de alteraciones matemáticamente confiable.

7.3. Respuestas del Sistema de Verificación

Condición	Alerta Visual	Mensaje
$H' = H \wedge$ firma válida $\wedge$ vigente	Verde	"Documento auténtico y vigente"
$H' = H \wedge$ firma válida $\wedge$ expirado	Amarillo	"Documento auténtico pero expirado"
$H' \neq H$	Rojo	"Documento alterado – integridad comprometida"

Folio inexistente	Rojo	"Folio no encontrado en el sistema"
Firma inválida	Rojo	"Firma digital no verificada – posible falsificación"

8. Módulo de Gestión de Beneficiarios

8.1. Motivación y Contexto

El módulo de beneficiarios centraliza el registro de las personas migrantes atendidas por Casa Monarca, funcionando como una bóveda cifrada de registros humanitarios. Su diseño parte de dos premisas: máxima protección de la privacidad de los beneficiarios y trazabilidad completa de los cambios en sus registros.

8.2. Modelo de Datos Encriptado

El modelo Beneficiario en `app/models.py` define todos los campos sensibles con el tipo personalizado `EncryptedText`, garantizando cifrado AES-256 en reposo:

```
class Beneficiario(Base):
    __tablename__ = "beneficiarios"

    id: Mapped[int] # Clave primaria, en claro
    nombre_completo: Mapped[str] # EncryptedText – cifrado AES-256
    pais_origen: Mapped[str] # EncryptedText – cifrado AES-256
    fecha_ingreso: Mapped[datetime] # DateTime, en claro (índice temporal)
    area: Mapped[str] # EncryptedText – cifrado AES-256
    area_lookup: Mapped[str] # HMAC-SHA256 – índice de búsqueda
    status: Mapped[str] # EncryptedText – cifrado AES-256
    notas: Mapped[str] # EncryptedText – cifrado AES-256
    created_by_user_id: Mapped[int] # FK a users, en claro (trazabilidad)
    created_at: Mapped[datetime] # DateTime, en claro
    updated_at: Mapped[datetime] # DateTime, en claro
```

La innovación clave es el campo `area_lookup`: mientras el campo `area` está completamente cifrado, `area_lookup` almacena el digesto HMAC-SHA256 del valor en claro, permitiendo consultas de igualdad sobre el campo cifrado sin necesidad de descriptarlo:

```
@validates("area")
def _sync_area_lookup(self, _key, value):
    self.area_lookup = database_crypto.lookup_digest(value.strip())
    return value.strip()
```

8.3. Arquitectura de Cifrado con Consultas (Searchable Encryption)

El motor `DatabaseCrypto` en `app/crypto.py` implementa una forma de Cifrado Buscable (Searchable Encryption) mediante dos mecanismos complementarios:

8.3.1. Cifrado de Texto (Fernet AES-256)

```
class DatabaseCrypto:
    token_prefix = "enc:v1:"
    lookup_prefix = "idx:v1:"

    def __init__(self, secret: str) -> None:
        encryption_material = sha256(f"{secret}:cipher") # 32 bytes
        lookup_material = sha256(f"{secret}:lookup") # 32 bytes
```



```
self._fernet = Fernet(b64(encryption_material))
self._lookup_key = lookup_material
```

Los tokens cifrados siguen el esquema versionado `enc:v1:{token_fernet}`, permitiendo migraciones futuras de algoritmo sin romper registros existentes (el prefijo actúa como discriminador de versión).

8.3.2. Digestos HMAC-SHA256 para Índices

Los digestos de búsqueda siguen el esquema `idx:v1:{hmac_hex}`. El prefijo `idx:v1:` documenta la versión del esquema de lookup y permite detectar registros sin digesto (pendientes de migración):

```
digesto = HMAC-SHA256(lookup\_key, valor\_en\_claro)
```

La clave `lookup_key` es derivada de la `session_secret` del sistema mediante SHA-256, garantizando que incluso si un atacante accede a la base de datos, los digestos de lookup son indistinguibles de ruido aleatorio sin conocer el secreto del sistema.

8.4. Control de Acceso por Rol en el Módulo de Beneficiarios

Rol	Crear	Leer	Actualizar estado	Eliminar
ADMIN	✓	✓ (todos)	✓	✓
COORDINADOR	✓	✓ (todos)	✓ (área propia)	×
OPERATIVO	✓	✓ (todos)	×	×
VOLUNTARIO	✓	×	×	×

8.5. Ciclo de Vida de un Registro de Beneficiario

Los registros de beneficiarios siguen un flujo de estados que refleja el proceso de atención:

nuevo → en_revision → canalizado → activo

- **nuevo:** registrado por voluntario u operativo, pendiente de revisión por coordinador.
- **en_revision:** el coordinador del área está evaluando el caso.
- **canalizado:** el caso fue derivado a la instancia u organización adecuada.
- **activo:** el beneficiario está siendo atendido activamente por Casa Monarca.

8.6. Exportación con Integridad Criptográfica

Los reportes PDF exportados desde el sistema incluyen automáticamente una huella SHA-256 incrustada en el pie del documento, implementada en `app/main.py`:

```
def _pdf_report(title, subtitle, col_headers, col_widths, rows, generated_by):
    # Construye las líneas de datos en texto plano
    data_lines = ["\t".join(col_headers)]
    for row in rows:
        data_lines.append("\t".join(str(c) for c in row))

    # Calcula el hash SHA-256 del contenido tabular completo
    sha256 = hashlib.sha256("\n".join(data_lines).encode()).hexdigest()

    # ... construcción del PDF con fpdf2 ...

    # Incrusta el hash en el pie del documento
    pdf.multi_cell(0, 5,
        f"Generado por: {generated_by} | {generated_at}\n"
        f"Integridad SHA-256: {sha256}"
    )
```

Esta técnica garantiza que el receptor de un reporte exportado pueda verificar manualmente que el contenido no fue alterado después de la generación. El hash cubre la totalidad de los datos tabulares (encabezados + filas), incluyendo separadores de campo (\t) y de línea (\n), por lo que cualquier modificación al contenido del reporte invalida el hash.

8.7. Datos de Demostración

El sistema incluye 10 registros de beneficiarios realistas que se cargan automáticamente si `SEED_DEMO_DATA=true`. Los registros cubren múltiples países de origen (Honduras, El Salvador, Venezuela, Guinea Conakry, Siria, Mali, Guatemala, Nicaragua), todas las áreas de atención y todos los estados del ciclo de vida, permitiendo demostrar el sistema sin comprometer datos reales de personas migrantes.

9. Sistema de Notificaciones y Salud del Certificado

9.1. Motivación

Los certificados X.509 tienen fecha de expiración. Un administrador o coordinador que no es alertado a tiempo de que su certificado está próximo a expirar perderá su acceso al portal sin advertencia, interrumpiendo operaciones críticas de atención humanitaria. El sistema de notificaciones aborda este riesgo de forma proactiva.

9.2. Arquitectura del NotificationService

El servicio NotificationService en `app/services.py` implementa cuatro funciones principales:

9.2.1. Verificación de Expiración de Certificados

`EXPIRY_WARNING_DAYS = 30` # días antes de expiración para alertar
`MAX_LOGIN_ATTEMPTS = 10` # intentos antes de bloqueo

```
@classmethod
def check_expiring_certificates(cls, db: Session) -> None:
    cutoff = datetime.utcnow() + timedelta(days=cls.EXPIRY_WARNING_DAYS)
    # Obtiene usuarios activos con certificados próximos a expirar
    for user in usuarios_activos_con_certificado:
        if user.certificate_not_after < cutoff:
            # Evita notificaciones duplicadas en ventana de 24 horas
            existing = buscar_notificacion_reciente(user.id, "cert_expiring_soon")
            if not existing:
                cls.create(db, type="cert_expiring_soon", user_id=user.id,
                    title="Certificado próximo a expirar",
                    message=f"El certificado de {user.full_name} expira en menos "
                        f"de {cls.EXPIRY_WARNING_DAYS} días...")
```

9.2.2. Verificación de Expiración de Cuentas

Análogamente, el sistema verifica el campo `end_date` de todas las cuentas activas y genera notificaciones tipo `user_expiring_soon` con la misma ventana de 30 días y anti-duplicación de 24 horas.

9.2.3. Notificaciones de Bloqueo por Intentos Fallidos

Cuando PasswordLoginService O SignatureLoginService detectan que una cuenta alcanzó el umbral de `MAX_LOGIN_ATTEMPTS = 10` intentos fallidos, genera automáticamente una notificación tipo `login_blocked` dirigida al administrador:

```

NotificationService.create(
    db, type="login_blocked", user_id=user.id,
    title=f"Cuenta bloqueada: {user.full_name}",
    message=f"La cuenta de {user.full_name} ({user.email}) fue bloqueada tras "
            f"{MAX_LOGIN_ATTEMPTS} intentos fallidos. "
            f"Desbloquear desde Gestionar cuenta."
)

```

9.2.4. Indicadores Visuales en la Interfaz (“Salud del Certificado”)

La sección “Mi Cuenta” del portal muestra el estado de salud del certificado del usuario mediante un sistema de semáforos basado en el tiempo restante hasta la expiración:

Estado	Color	Condición
Activo	Verde	Certificado válido con más de 30 días restantes
Próximo a expirar	Amarillo	Entre 8 y 30 días para la expiración
Crítico	Naranja	7 días o menos para la expiración
Expirado	Rojo	La fecha certificate_not_after fue superada
Pendiente	Gris	Usuario sin certificado emitido aún

9.3. Buzón de Notificaciones

El panel lateral del portal incluye un buzón de notificaciones que muestra: alertas de certificados próximos a expirar, alertas de cuentas próximas a expirar, notificaciones de cuentas bloqueadas (solo administradores) y documentos pendientes de co-firma (coordinadores).

El contador de notificaciones no leídas se actualiza en cada carga de página al consultar `NotificationService.list_unread(db)`. El administrador puede marcar todas como leídas desde el Panel de Administración.

10. Sistema de Recuperación de Emergencia – Administrador Espejo

10.1. Problema de la Llave Única

El Principio de Entrega Única garantiza No Repudio pero introduce un riesgo operativo: si el administrador único pierde sus credenciales, queda bloqueado fuera del sistema indefinidamente, sin posibilidad de gestionar usuarios, certificados ni emergencias.

El Sistema de Administrador Espejo (Mirror Admin) resuelve este problema manteniendo en todo momento un administrador de respaldo en estado `revoked` que puede ser activado de emergencia.

10.2. Arquitectura del Patrón Espejo

Estado normal:

Admin Principal (ACTIVO) ← gestiona el sistema
 Admin Espejo (REVOCADO) ← en espera, sin acceso

Estado de emergencia:

Admin Principal (REVOCADO) ← acceso bloqueado
 Admin Espejo (ACTIVO) ← toma el control

El invariante central del sistema es: nunca puede haber dos administradores activos simultáneamente. La operación de activación del espejo funciona revocando al principal y activando al espejo en la misma transacción de base de datos.

10.3. Implementación del AdminRecoveryService

```
class AdminRecoveryService:
```

```
    @staticmethod
    def activate_mirror(db: Session, actor: User) -> None:
        """Activa el admin espejo, revocando al principal."""
        primary = get_primary_admin(db)
        backup = get_backup_admin(db)

        if not backup or backup.status == "active":
            raise ValueError("No existe admin espejo disponible")

        # Transacción atómica: revocar principal + activar espejo
        primary.status = "revoked" # bloquea inmediatamente al principal
        backup.status = "active" # el espejo toma el control

        # El principal pierde su contraseña; solo el espejo puede operar
        primary.password_hash = None

        # Auditoría forense
        AuditService.log(db, "admin_recovery_activated", actor.id,
                        target_user_id=backup.id, ...)

        db.commit()
```

10.4. Sincronización del Admin Espejo

El BootstrapService garantiza que siempre exista un admin de respaldo. Al iniciar el sistema, verifica la existencia del admin espejo. Si el admin espejo no tiene contraseña (p. ej., tras una regeneración), la asigna con la contraseña de demostración documentada por separado, nunca visible en el historial de la cuenta principal. El admin espejo está marcado con `is_backup_admin = True` en la base de datos, lo que impide que accidentalmente se le asignen los mismos certificados que al principal.

10.5. Flujo de Recuperación de Emergencia

1. El administrador principal detecta que sus credenciales están comprometidas o que perdió acceso a sus archivos `.pem`.
2. El administrador principal (o cualquier usuario con acceso físico al panel de administración) activa el botón **“Activar admin espejo”**.
3. El sistema ejecuta la transacción atómica: revocar principal → activar espejo.
4. El admin espejo entra al sistema con sus credenciales (previamente entregadas de forma segura fuera de banda) y procede a generar nuevas credenciales para el administrador principal, completando el ciclo de recuperación.

11. Gestión Expandida de Usuarios – Panel de Administración

11.1. Ciclo de Vida Completo del Usuario

La segunda etapa completa el ciclo de vida de identidades definido en la literatura de gestión de identidades (Oracle Corporation, 2024):

Operación	Descripción	Endpoint
Alta	Creación de cuenta con generación de certificado X.509	POST /ui/users

Activación	Cambio de estado pending → active	POST /ui/users/{id}/status
Cambio de rol	Actualización del rol y re-emisión de certificado	POST /ui/users/{id}/role
Cambio de vigencia	Actualización de end_date y extensión del certificado	POST /ui/users/{id}/expiration
Revocación	Cambio de estado active → revoked de emergencia	POST /ui/users/{id}/status
Desbloqueo	Liberar cuenta bloqueada por intentos fallidos	POST /ui/users/{id}/unlock
Re-emisión	Generar nuevas credenciales criptográficas para un usuario	POST /ui/users/{id}/certificate
Baja	No implementada como eliminación; el estado revoked es permanente	N/A

11.2. Vista del Certificado X.509

El sistema provee una página dedicada para la inspección visual del certificado X.509 de cualquier usuario autorizado, accesible en GET /ui/users/{user_id}/certificate/view:

```
def render_certificate_page(title, summary, back_href):
    # summary incluye:
    # - subject (CN, 0)
    # - issuer (CN, 0)
    # - serial_number (hexadecimal)
    # - not_valid_before_utc, not_valid_after_utc
    # - fingerprint_sha256 (huella SHA-256 del certificado)
    # - signed_by, signed_with, signer_verified_with
```

Los campos expuestos permiten al administrador verificar manualmente la cadena de confianza sin necesidad de herramientas externas como openssl x509 -text.

11.3. Exportación del Directorio de Usuarios

Los coordinadores y administradores pueden descargar en PDF el directorio completo de usuarios registrados. El reporte incluye: nombre completo, correo electrónico, rol, estado y fecha de creación. El pie de página del PDF incrusta el SHA-256 del contenido tabular, garantizando que el directorio descargado no fue alterado tras su generación.

```
GET /ui/users/export.pdf
→ Verifica: actor.role in (ADMIN, COORDINADOR) AND actor.status == "active"
→ Genera PDF con _pdf_report(...)
→ Header: Content-Disposition: attachment; filename=usuarios.pdf
```

11.4. Exportación de la Bitácora de Auditoría

Los administradores pueden exportar en PDF los últimos 200 eventos de auditoría:

```
GET /ui/audit-logs/export.pdf
→ Verifica: is_active_admin(actor)
→ Columnas: Fecha | Evento | Acción | Resultado | IP
→ Footer: SHA-256 del contenido tabular completo
```

La combinación de bitácora de auditoría exportable + SHA-256 incrustado en el PDF permite a las autoridades competentes verificar la integridad del reporte forense sin necesidad de acceder directamente a la base de datos.

12. Motor DatabaseCrypto – Análisis Técnico Profundo

12.1. Derivación de Claves de Cifrado y Lookup

El motor DatabaseCrypto en `app/crypto.py` deriva dos claves independientes a partir del secreto del sistema mediante SHA-256:

```
encryption_material = sha256(f"{secret}:cipher".encode()).digest() # 32 bytes
lookup_material     = sha256(f"{secret}:lookup".encode()).digest() # 32 bytes
```

La separación de dominios mediante los sufijos `:cipher` y `:lookup` garantiza que incluso con el mismo secreto base, las claves para cifrado y para generación de digestos sean criptográficamente independientes. Un atacante que comprometa la clave de lookup no obtiene capacidad de descifrado, y viceversa.

12.2. Cifrado Fernet: Detalles de Implementación

Fernet (del paquete `cryptography`) es un esquema de cifrado simétrico autenticado que internamente combina AES-128 en modo CBC para el cifrado del contenido, HMAC-SHA256 para la autenticidad del token y por último, IV (Vector de Inicialización) aleatorio de 16 bytes generado por `os.urandom`.

El token Fernet resultante tiene la estructura:

Version (1B) | Timestamp (8B) | IV (16B) | Ciphertext (variable) | HMAC (32B)

Todo codificado en Base64 URL-safe. La versión del token (0x80) permite migraciones futuras del esquema de cifrado.

El sistema prefija adicionalmente todos los tokens con `"enc:v1:"` como discriminador de versión de la aplicación, independiente del versionado interno de Fernet. Esta decisión permite detectar campos sin cifrar (registros migrados) y diferenciarlos de campos cifrados en tiempo de desencriptación:

```
def is_encrypted(self, value: Any) -> bool:
    return isinstance(value, str) and value.startswith(self.token_prefix)

def decrypt_text(self, value: Any) -> str | None:
    if not self.is_encrypted(value):
        return value # Campo sin cifrar (pre-migración): retornar en claro
    token = value[len(self.token_prefix):]
    return self._fernet.decrypt(token.encode()).decode("utf-8")
```

12.3. Migración de Datos Legados

El `SchemaService` incluye lógica de migración para convertir campos en texto plano de versiones anteriores a campos cifrados con el esquema `enc:v1:`.

La migración lee todos los registros existentes de la tabla. Para cada registro, descifra el valor actual (que puede estar en texto plano si es una migración o ya cifrado si fue actualizado). Después, re-cifra con el esquema actual, actualiza el registro en la base de datos y actualiza el digesto de lookup si el campo tiene índice HMAC.

Este proceso garantiza compatibilidad hacia atrás sin pérdida de datos durante actualizaciones del sistema en producción.

13. Interfaz de Usuario – Diseño para Contextos Humanitarios

13.1. Principios de Diseño

La interfaz está diseñada con cinco principios guía adaptados al contexto humanitario de Casa Monarca:

1. *Cero dependencias externas*: toda la interfaz es HTML server-side rendering sin frameworks JavaScript. Esto garantiza que el sistema funcione en dispositivos con conectividad limitada y reduce la superficie de ataque frente a CDNs comprometidas o ataques de cadena de suministro.
2. *Semáforo de estados*: los estados críticos de usuarios y certificados se comunican mediante código de colores universalmente comprensible (verde = activo, amarillo = pendiente, rojo = revocado, gris = expirado), reduciendo errores operativos por parte de personal con baja alfabetización digital.
3. *Acciones ocultas por rol*: los botones de operaciones destructivas (Revocar, Eliminar, Cambiar Rol) solo se inyectan en el DOM si el token de sesión confirma el rol jerárquico adecuado. Un operativo que inspecciona el HTML de la página nunca verá los botones de admin.
4. *Drag-and-drop para archivos criptográficos*: las zonas de carga de archivos .pem permiten arrastrar y soltar, eliminando la necesidad de navegar por el sistema de archivos y reduciendo el riesgo de cargar el archivo incorrecto.
5. *Retroalimentación contextual*: mensajes de confirmación, error y advertencia se muestran directamente en el flujo de trabajo sin redirecciones innecesarias, usando el sistema de notice y error en los parámetros de URL.

13.2. Estructura de Navegación del Portal

El portal de usuario presenta una barra de navegación lateral con secciones dinámicas según el rol y estado del usuario activo:

Sección	ADMIN	COORD.	OPER.	VOL.
Mi Cuenta	✓	✓	✓	✓
Mis Credenciales	✓	✓	×	×
Beneficiarios	✓ (activo)	✓ (activo)	✓ (activo)	✓ (activo)
Directorio	✓	✓	×	×
Firmar Documentos	✓	✓	×	×
Verificar Documento	Público	Público	Público	Público
Panel de Admin	✓	×	×	×

13.3. Flujo de Autenticación Unificado

La pantalla de login (renderizada por `main.py` en `render_login_page`) integra ambas modalidades de autenticación en un solo formulario:

- Los campos **Archivo de acceso** y **Certificado de identidad** permanecen visibles para todos, pero con instrucciones contextuales que explican que son exclusivos para administradores y coordinadores.
- El campo **Contraseña** sirve tanto como contraseña de cuenta (voluntarios/operativos) como contraseña de desencriptación de la llave .pem (roles privilegiados).
- Si los campos de archivo están vacíos al enviar el formulario, el sistema intenta autenticación por contraseña; si los campos tienen archivos, activa el flujo criptográfico.

Esta decisión de diseño evita páginas de login separadas que podrían confundir al personal sobre qué modalidad usar, centralizando la lógica de bifurcación en el backend.

14. Configuración, Despliegue y Entorno de Producción

14.1. Variables de Entorno Críticas

La clase `Settings` en `app/config.py` (basada en `pydantic-settings`) define la configuración del sistema y aplica validaciones críticas:

Variable	Entorno	Descripción
ENVIRONMENT	prod	Activa validaciones de seguridad adicionales
SESSION_SECRET	prod	Secreto HMAC de sesiones; NUNCA usar el valor por defecto
DATABASE_ENCRYPTION_KEY	prod	Clave de cifrado Fernet; si vacío, deriva de SESSION_SECRET
DATABASE_URL	prod	URL PostgreSQL para producción
SEED_DEMO_DATA	dev	false en producción para evitar datos de demostración
BOOTSTRAP_ADMIN_EMAIL	prod	Correo del administrador inicial
BOOTSTRAP_ADMIN_PASSWORD	prod	Contraseña inicial del administrador

14.2. Guardián de Producción (Production Guard)

Si `ENVIRONMENT=production` y `SESSION_SECRET` conserva el valor por defecto de desarrollo ("cambia-esto-en-produccion-usa-variable-de-entorno"), el servidor se niega a arrancar:

```
if settings.environment == "production":
    assert settings.session_secret != "cambia-esto-en-produccion-...", \
        "SESSION_SECRET debe ser cambiado en producción"
```

Este guardián activo impide el despliegue accidental de una instancia de producción con secretos débiles, aplicando seguridad desde el diseño (*security by design*).

14.3. Soporte Multi-Base de Datos

El sistema soporta SQLite (desarrollo local, sin dependencias externas) y PostgreSQL (producción, con el driver `psycopg[binary]`). La URL de la base de datos se configura mediante `DATABASE_URL`:

```
# Desarrollo
database_url = "sqlite:///identity_demo.db"

# Producción
database_url = "postgresql+psycopg://user:pass@host/dbname"
```

El `SchemaService` detecta el dialecto activo y adapta la sintaxis DDL para migraciones de columnas y creación de índices, garantizando compatibilidad sin necesitar un sistema de migraciones externo como Alembic.

14.4. Containerización con Docker

El proyecto incluye un `Dockerfile` que empaqueta la aplicación como una imagen autocontenida:

```
FROM python:3.12-slim
WORKDIR /app
COPY pyproject.toml .
RUN pip install -e ".[prod]"
COPY app/ app/
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

El archivo `render.yaml` configura el despliegue automático en Render.com, con PostgreSQL administrado y variables de entorno secretas gestionadas por la plataforma.

15. Resultados de la Segunda Etapa

15.1. Funcionalidades Implementadas y Verificadas

Componente	Funcionalidad	Estado
Firma Digital de Documentos	SHA-256 + RSA-PSS-MGF1-SHA256, Folio único	Implementado
Co-firma Multi-actor	Flujo paralelo y secuencial con notificaciones	Implementado
Firma Masiva en Lote	Hasta 20 archivos por operación de lote	Implementado
Verificación Pública por Folio	Sin autenticación, consulta de metadatos	Implementado
Verificación por Archivo	Re-hash SHA-256 + verificación RSA-PSS	Implementado
Bóveda de Beneficiarios	CRUD con cifrado AES-256 y lookup HMAC	Implementado
Exportación con SHA-256	PDF con integridad criptográfica incrustada	Implementado
Notificaciones de Certificado	Alertas 30 días antes de expiración	Implementado
Notificaciones de Cuenta	Alertas 30 días antes de end_date	Implementado
Admin Espejo	Activación atómica de recuperación de emergencia	Implementado
Desbloqueo de Cuenta	Liberación manual de cuentas bloqueadas	Implementado
Exportación de Auditoría	PDF con últimos 200 eventos + SHA-256	Implementado
Exportación de Directorio	PDF de usuarios con SHA-256	Implementado
Exportación de Beneficiarios	PDF de registros humanitarios con SHA-256	Implementado
Vista de Certificado X.509	Inspección de extensiones y fingerprint SHA-256	Implementado

15.2. Casos de Prueba Adicionales de la Segunda Etapa

Caso	Descripción	Resultado Esperado	Estatus
Firma individual	Coord. firma un PDF de 1 MB	Hash + firma + folio generados; doc no almacenado	Exitoso
Co-firma secuencial	Dos coordinadores firman en orden	Segundo no puede firmar hasta que el primero firma	Exitoso
Verificación válida	Verificar archivo original con folio correcto	Documento auténtico y vigente	Exitoso
Verificación alterada	Verificar archivo modificado (1 bit cambiado)	“Documento alterado” – hash no coincide	Exitoso
Registro beneficiario	Voluntario registra persona con área y país	Registro guardado cifrado; solo hash en índice	Exitoso
Actualización estado	Coordinador cambia estado de nuevo a en_revision	Estado actualizado; lookup HMAC actualizado	Exitoso
Exportar PDF auditoría	Admin exporta últimos 200 eventos	PDF generado con SHA-256 en footer	Exitoso
Notif. cert. expiración	Certificado dentro de 30 días para expirar	Notificación creada; no duplicada en 24h	Exitoso

Activar admin espejo	Admin principal activa el espejo	Principal revocado atómicamente; espejo activo	Exitoso
Firma masiva 20 archivos	Coord. firma 20 constancias en un lote	20 folios únicos generados; 20 eventos en auditoría	Exitoso
Desbloqueo de cuenta	Admin desbloquea cuenta tras 10 intentos fallidos	login_attempts = 0; acceso restaurado	Exitoso
Importar certificado ajeno en co-firma	Coord. intenta firmar con certificado de otro usuario	Firma rechazada: SAN no corresponde al actor	Exitoso

15.3. Métricas Técnicas del Sistema Completo

Métrica	Valor
Tablas de base de datos	7 (users, roles, permissions, role_permissions, audit_logs, beneficiarios, notifications, system_secrets)
Campos con cifrado AES-256	≥ 40 campos a través de todas las tablas
Campos con índice HMAC-SHA256	8 campos (email_lookup, status_lookup, area_lookup, type_lookup, code_lookup, key_lookup)
Longitud de llave RSA	2048 bits
Iteraciones PBKDF2	120,000
Duración máxima de sesión	8 horas (28,800 segundos)
Tamaño máximo de archivo (firma)	20 MB
Máximo de archivos por lote	20
Intentos de login antes de bloqueo	10
Días de advertencia antes de expiración	30

16. Conclusiones Finales

La implementación completa del sistema para Casa Monarca demuestra que es posible diseñar una plataforma de gestión de identidades y firma digital que satisfaga simultáneamente tres requisitos sumamente importantes:

Primero, la alta seguridad criptográfica. Se utilizó RSA-2048 con PSS para firmas y autenticación; SHA-256 para integridad de documentos y exportaciones; AES-256 Fernet para cifrado en reposo; PBKDF2-HMAC-SHA256 con 120,000 iteraciones para contraseñas; HMAC-SHA256 para índices de búsqueda sobre campos cifrados; y cookies de sesión firmadas con HMAC acotadas a 8 horas.

Segundo, la usabilidad humanitaria. Para nosotros era importante crear un sistema que pudiera ser verdaderamente implementado por los administradores de Casa Monarca, sin sacrificar la complejidad técnica que garantiza una seguridad real. Por esta razón, la interfaz server-side sin frameworks externos funciona en dispositivos con conectividad limitada; el semáforo de estados permite auditorías visuales rápidas; la bifurcación de autenticación evita barreras para usuarios de baja habilidad técnica; los mensajes de error son contextuales y en español.

Por último, el cumplimiento legal. Los Derechos ARCO están materializados en funciones concretas del sistema (visualización, corrección, cancelación/oposición mediante revocación); la bitácora de auditoría inmutable satisface el principio de responsabilidad y trazabilidad de la Ley Federal de Protección de Datos Personales.

El Principio de Cero Almacenamiento en el módulo de firma garantiza que incluso ante un compromiso total de la base de datos, los documentos confidenciales de los beneficiarios migrantes permanecen protegidos: un atacante solo obtendría hashes SHA-256 y firmas RSA, sin ningún contenido legible.

El Sistema de Administrador Espejo cierra el ciclo de continuidad operativa: ningún escenario de pérdida de credenciales puede dejar a Casa Monarca sin acceso a su sistema de gestión de identidades, garantizando que la misión humanitaria nunca sea interrumpida por un incidente de seguridad.

16.1. Trabajo Futuro

- **Blockchain en bitácoras:** migrar `audit_logs` a una red de nodos permitida (Hyperledger Fabric) para elevar la inmutabilidad forense a nivel criptográfico distribuido.
- **WebAuthn / FIDO2:** reemplazar los archivos `.pem` manuales con autenticación biométrica nativa de dispositivos móviles, eliminando la necesidad de custodiar archivos físicos.
- **CRL / OCSP:** implementar listas de revocación de certificados en tiempo real, reemplazando la validación lógica basada en `end_date` por el protocolo OCSP estándar de la industria.
- **Cifrado Homomórfico Parcial:** evolucionar el esquema de cifrado de campos para permitir búsquedas analíticas y ordenamientos sobre datos cifrados sin descifrarlos en RAM, neutralizando ataques de volcado de memoria (*memory dumping*).
- **Integración con HSM:** migrar el almacenamiento de la llave privada de la CA raíz desde la tabla `system_secrets` hacia un Hardware Security Module (HSM) para protección de material criptográfico de alta sensibilidad en entornos de producción críticos.
- **Módulo de Reportes Estadísticos:** agregar dashboards de métricas de atención (beneficiarios por área, por país, por mes) con exportación cifrada para transparencia institucional ante organismos financiadores.

17. Bibliografía Completa

Firma-e. (2014, octubre 14). *Pilares de la Seguridad de la Información: confidencialidad, integridad y disponibilidad*. <https://www.firma-e.com/blog/pilares-de-la-seguridad-de-la-informacion>

IBM. (2026). *¿Qué es la gestión del ciclo de vida de los datos?* <https://www.ibm.com/mx-es/think/topics/data-lifecycle-management>

Oracle Corporation. (2024). *Lifecycle for Managing Users*. <https://docs.oracle.com/es-ww/iaas/Content/Identity/users/lifecycle-managing-users.htm>

National Institute of Standards and Technology. (2022). *FIPS PUB 186-5: Digital Signature Standard (DSS)*. U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.186-5>

Bellare, M., & Rogaway, P. (1996). *The Exact Security of Digital Signatures – How to Sign with RSA and Rabin*. Advances in Cryptology – EUROCRYPT '96, 399–416.

Rescorla, E. (2018). *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc8446>

Moriarty, K., et al. (2016). *PKCS 8: Private-Key Information Syntax Specification Version 1.2*. RFC 5958. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc5958>

Cooper, D., et al. (2008). *Internet X.509 Public Key Infrastructure Certificate and CRL Profile*. RFC 5280. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc5280>

Instituto Nacional de Transparencia, Acceso a la Información y Protección de Datos Personales (INAI). (2010). *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación.

Python Software Foundation. (2024). *hashlib – Secure hashes and message digests*. <https://docs.python.org/3/library/hashlib.html>

The cryptography authors. (2024). *cryptography: Cryptographic recipes and primitives for Python*. <https://cryptography.io/en/latest/>

Tiangolo, S. (2024). *FastAPI – Modern, fast (high-performance), web framework for building APIs*. <https://fastapi.tiangolo.com/>

Ronacher, A. (2024). *itsdangerous – Safely pass data to untrusted environments*. <https://itsdangerous.palletsprojects.com/>

18. Anexos

18.1. Tabla de casos de prueba

Caso	Descripción	Resultado	Evidencia
Alta de usuario válida	Admin crea usuario con nombre, correo único, rol, fecha opcional y contraseña .p12.	Usuario creado correctamente, certificado emitido, redirección al panel.	<u>Alta de usuario válida</u>
Alta con correo duplicado	Intentar crear usuario con correo ya existente.	Rechazo de la operación con error de validación.	<u>Alta con correo duplicado</u>
Alta con nombre vacío	Crear usuario con nombre en blanco o espacios.	Rechazo con error de validación.	<u>Alta con nombre vacío</u>
Alta con contraseña .p12 vacía	Crear usuario sin contraseña para .p12.	Rechazo con error de validación.	<u>Alta con contraseña .p12 vacía</u>
Alta por usuario sin permiso	Usuario no admin intenta crear usuario.	Acceso no autorizado.	<u>Alta por usuario sin permiso</u>
Activar usuario pending	Admin cambia estado de pending a active.	Estado actualizado a active.	<u>Activar usuario pending</u>
Revocar usuario activo	Admin cambia estado de active a revoked.	Estado actualizado a revoked.	<u>Revocar usuario activo</u>
Reactivar usuario revocado	Admin activa usuario que estaba revoked.	Estado pasa a active con acción de reactivación.	<u>Reactivar usuario revocado</u>
Login demo admin/admin	Ingresar con credenciales de by-pass demo.	Login exitoso al dashboard admin.	<u>Login demo admin/admin</u>
Login .p12 exitoso	Correo válido + p12 correcto + contraseña correcta.	Login exitoso con identidad verificada.	<u>Login .p12 exitoso</u>
Login sin archivo .p12	Intentar login normal sin adjuntar archivo.	Rechazo con mensaje de archivo requerido.	<u>Login sin archivo .p12</u>
Cambiar expiración futura	Admin establece end date futura.	Fecha guardada correctamente.	<u>Cambiar expiración con fec.</u>
Cambiar expiración pasada	Admin intenta guardar expiración en fecha pasada.	Rechazo con error de validación.	<u>Cambiar expiración con fec...</u>
Usuario expira auto.	Usuario activo con fecha vencida al listar usuarios.	Estado cambia automáticamente a expired.	<u>Usuario expira automática...</u>
Reactivar por vigencia	Usuario expired recibe nueva fecha futura.	Estado vuelve a active.	<u>Reactivar por vigencia</u>
Cambio de rol válido	Admin cambia rol de un usuario.	Rol actualizado correctamente.	<u>Cambio de rol válido</u>
Login contraseña incorrecta	Subir .p12 válido con contraseña incorrecta.	Login rechazado.	<u>Login con contraseña incorrecta</u>
Login usuario no activo	Usuario en pending/revoked/expired intenta login.	Login rechazado por estado.	<u>Login con usuario no activo</u>
Certificado ajeno	Correo A + .p12 de usuario B.	Login rechazado por no correspondencia.	<u>Login con certificado de otro usuario</u>
Ver certificado propio	Usuario consulta vista/descarga de su certificado.	Acceso permitido.	<u>Ver certificado propio</u>
Auditoría admin	Admin consulta bitácora.	Respuesta correcta con eventos recientes.	<u>Consultar auditoría como administrador</u>
Usuario inexistente	Usuario que no existe intenta hacer login.	Usuario no registrado.	<u>Login usuario inexistente</u>
Admin revocado	Admin revocado intenta login tras activación de admin espejo.	Contraseña incorrecta.	<u>Archivo Admin Revocado</u>

Tabla 2: Evidencia de casos de pruebas del sistema (elaboración propia)